

V0088

Advanced Information Security

高级信息安全

Instructor: **Prof. Xiaohong Yu**

Graduate School of Information and Communication Technology,
Ajou University, Korea

软件安全

- 软件安全
 - 软件缺陷——缓冲区溢出等。
 - 恶意软件——病毒、蠕虫等。
- 软件逆向工程
- 数字版权管理

为什么选择软件?

- 为什么软件对安全的重要性不亚于密码学、访问控制和安全协议?
- 几乎所有信息安全功能都是在软件中实现的。
- 如果您的软件遭受攻击，您的安全防护就会失效。
 - 无论加密技术的强度、访问控制或协议如何

软件漏洞 and 恶意软件

- “冲击波”蠕虫 (Blaster Worm, Lovsan, MSBlast)
 - 微软漏洞: DCOM RPC 缓冲区溢出漏洞。
 - RPC服务在处理网络数据包时, 没有正确检查缓冲区边界, 导致栈缓冲区溢出
 - 导致电脑不断重启、网络拥塞
- WannaCry 勒索软件
 - 加密你的文件, 然后勒索比特币
 - 英国 NHS 医疗系统: 停诊, 手术取消, 无法查看病例
 - 利用Windows协议里EternalBlue远程缓冲区溢出漏洞, 发送网络包, 自动传播
 - 软件漏洞直接影响现实社会运行

Oops, your files have been encrypted!

软件问题

普通用户：爱丽丝和鲍勃

- ❑ 意外发现漏洞和缺陷
- ❑ 讨厌糟糕的软件.....
- ❑ 但他们学会了接受这一切。
- ❑ 必须让糟糕的软件也能运行。

攻击者：特鲁迪

- 积极寻找漏洞和缺陷
- 喜欢劣质软件.....
- 并试图让它做出不理智的行为
- 通过恶意软件攻击系统

软件复杂性

- “Complexity is the enemy of security”, Paul Kocher, the “father of SSL”

System	Lines of Code (LOC)
Netscape	17 million
Space Shuttle	10 million
Linux kernel 2.6.0	5 million
Windows XP	40 million
Mac OS X 10.4	86 million
Boeing 777	7 million

- 现在一辆新车的芯片容量比之前阿波罗号宇航员登月所需的芯片容量还要多

代码量与错误

- 保守估计：5 个错误/10,000 行代码
- 算算：
 - 典型计算机：3000 个可执行文件，每个文件 10 万行代码。
 - 保守估计：每个可执行文件约 **50 个错误**。
 - 这意味着每台计算机大约有 **15 万个漏洞**。
 - 所以，一个拥有3万个节点的网络有45亿个漏洞。
 - 或许只有 10% 的漏洞是安全关键漏洞，而其中只有 10% 的漏洞可以被远程利用。
 - 那么“仅仅”存在 4500 万个严重安全漏洞！

软件安全

- 程序缺陷（非故意）
 - 缓冲区溢出
 - 不完整验证
 - 竞态条件
- 恶意软件（故意）
 - 病毒
 - 蠕虫
 - 其他类型的恶意软件

软件缺陷

- 错误是指编程失误。
 - 人非圣贤，孰能无过。
- 错误可能导致状态不正确：故障
 - 故障是程序内部的。
- 故障可能导致系统失效，即系统偏离其预期行为。
 - 故障是可以从外部观察到的。



代码示例

```
char array[10];  
for(i = 0; i < 10; ++i)  
    array[i] = `A`;  
array[10] = `B`;
```

- ❑ 此程序有**错误**
- ❑ 此错误可能导致**故障**
 - 内部状态错误
- ❑ 如果发生故障，可能会导致**失败**。
 - 程序运行异常(外部)
- ❑ 对以上所有情况都使用“**缺陷**”一词。

安全软件

- 在软件工程中，要努力确保程序能够实现预期功能。
- 安全的软件工程要求软件能够按预期运行.....（只做该做的事）
- 仅此而已
- 绝对安全的软件？
 - 任何地方都没有绝对安全的软件。不可能
- 我们如何管理软件风险？

程序缺陷

- 程序缺陷并非**故意造成**。
 - 但仍然可能造成安全风险
- 我们将考虑三种类型的缺陷。
 - 缓冲区溢出（堆栈溢出）
 - 不完全验证
 - 竞态条件
- 这些是最常见的缺陷

缓冲区溢出

- 用户在网页表单中输入数据
- 网页表单已发送到服务器
- 服务器将数据写入名为buffer 的数组，而不检查输入数据的长度。
- 数据“溢出”缓冲区
 - 这种溢出可能使攻击成为可能。
 - 如果情况属实，任何能够访问该网页表单的人都可以发起攻击。

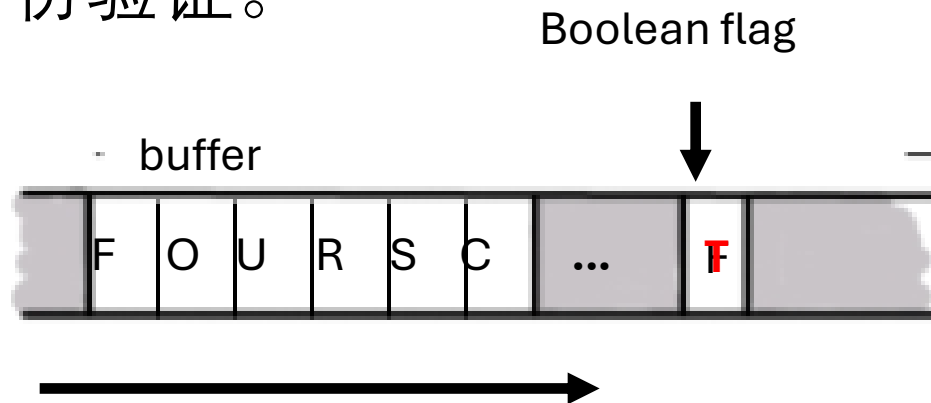
缓冲区溢出

```
int main() {  
    int buffer[10];  
    buffer[20] = 37;  
}
```

- **问：** 代码执行时会发生什么？
- **答：** 取决于内存地址“ buffer[20] ”处存储的内容。
 - 可能会覆盖**用户**数据或代码
 - 可能会覆盖**系统**数据或代码
 - 或者程序本身也能正常运行。

简单的缓冲区溢出

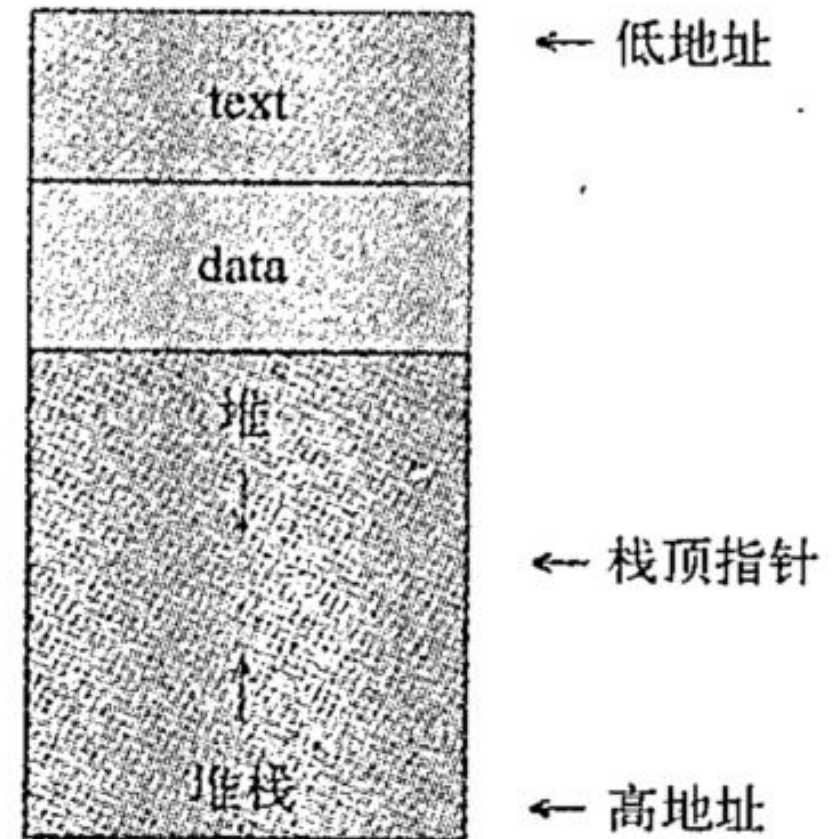
- 考虑使用**布尔标志**进行身份验证
- 缓冲区溢出可能导致标志被覆盖，从而使任何人都能进行身份验证。



- 在某些情况下，特鲁迪未必像这个例子中那样幸运。

内存组织结构

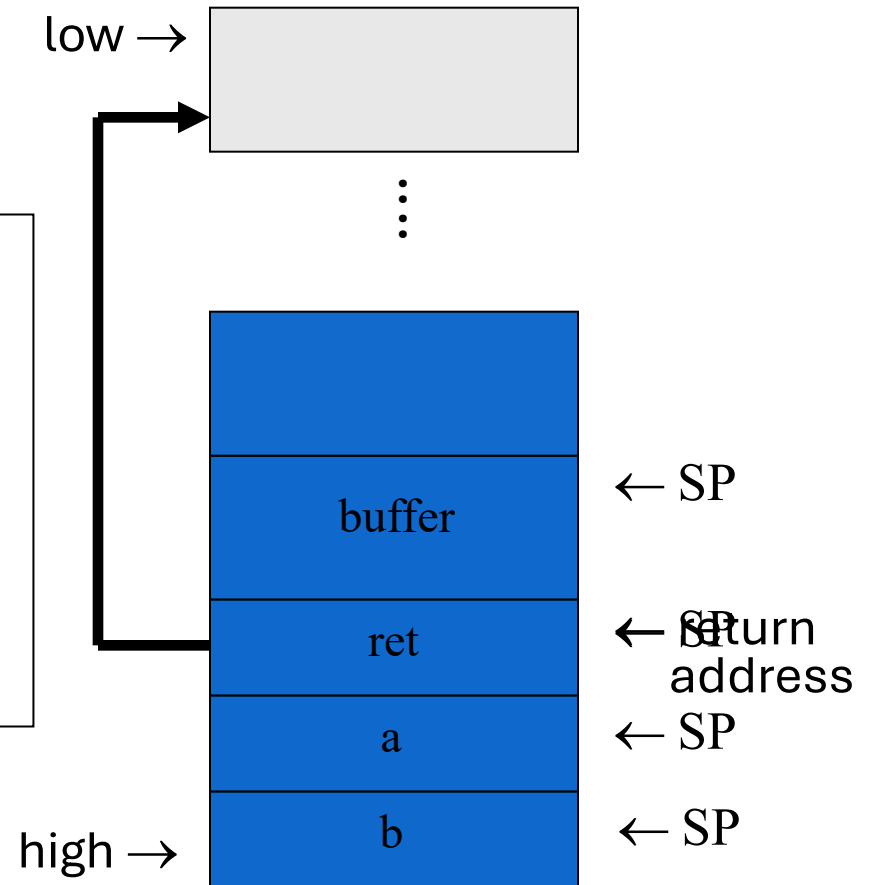
- Text: 存放代码
- Data: 存放静态数据
- 堆: 存放动态数据, 存储由低地址到高地址
- 堆栈: 处理器便签, 存储由高地址到低地址
 - 动态变量
 - 函数参数
 - 函数返回地址



简单堆栈正常示例

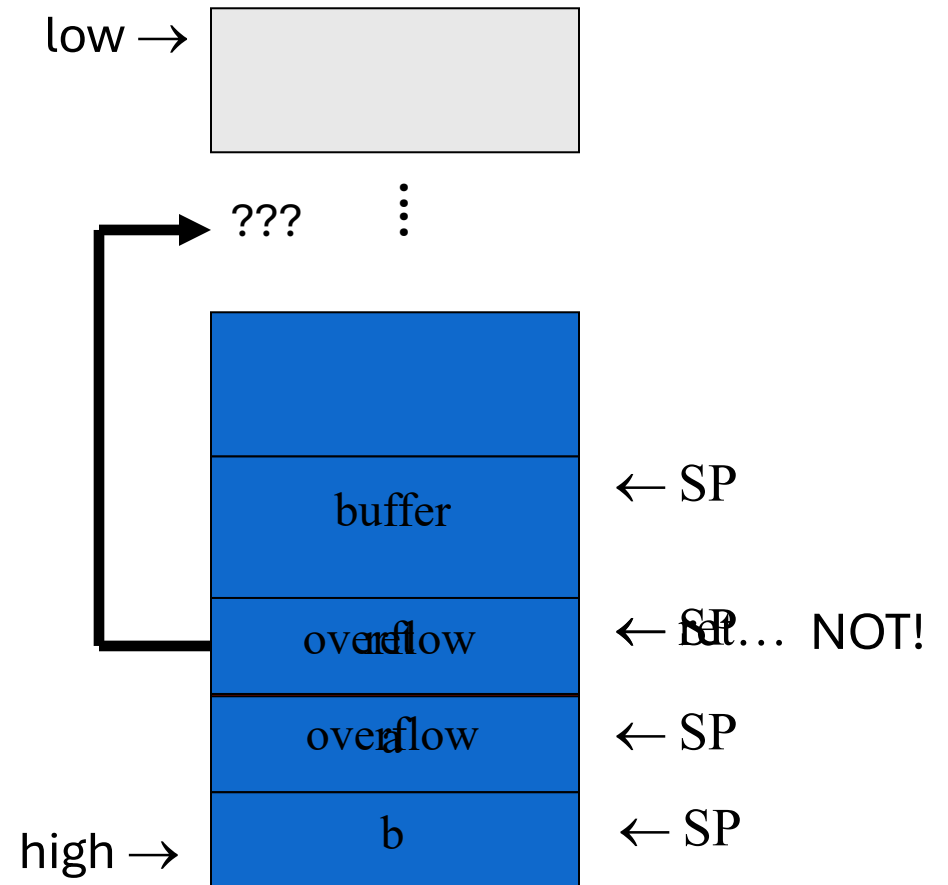
- 由主函数main进入（主函数在低地址）
- 执行func函数（

```
void func(int a, int b) {  
    char buffer[10];  
}  
void main() {  
    func(1, 2);  
}
```



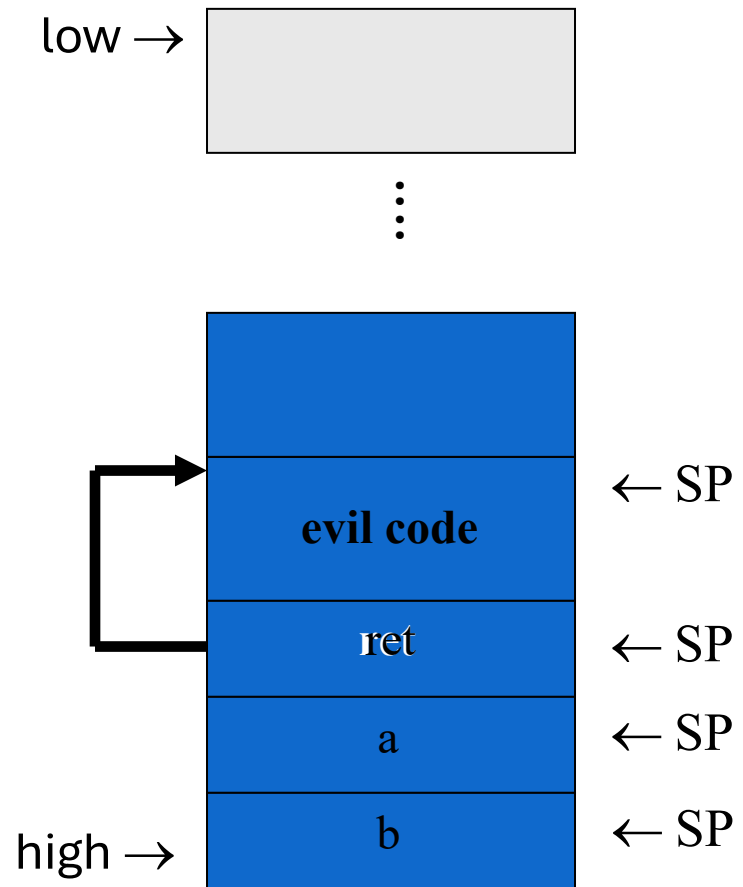
堆栈溢出缺陷示例

- ❑ 缓冲区溢出会发生什么?
- ❑ 程序“返回”错误位置
- ❑ 可能发生冲突和碰撞



堆栈溢出缺陷示例

- ❑ 攻击者可能不满足于只让代码崩溃,
- ❑ **代码注入**
- ❑ 攻击者可以运行她自己的代码
...
- ...在你的机器上



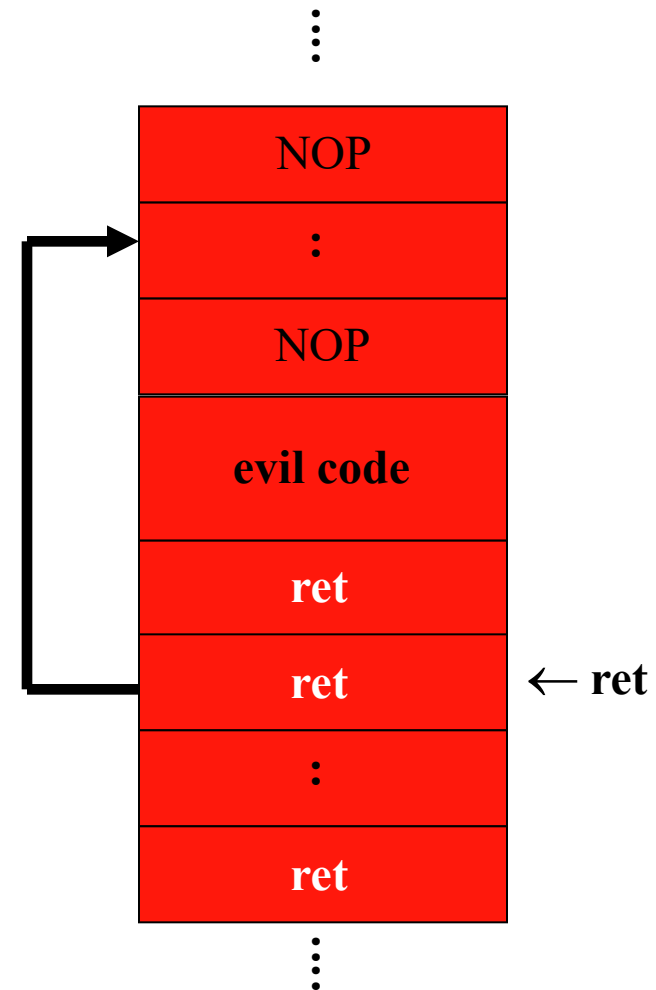
堆栈溢出缺陷示例

❑ 攻击者可能不知道.....

- 1) 恶意代码的地址
- 2) **ret**在堆栈上的位置

❑ 解决方案

- 1) 在恶意代码前加上 NOP“着陆点”
- 2) 多次插入**ret**



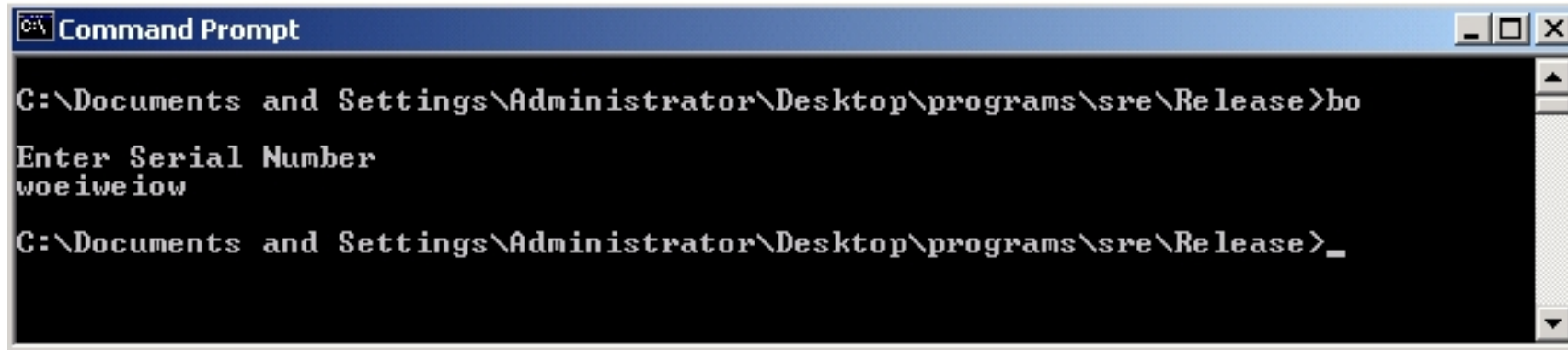
堆栈溢出总结

- 代码中必须存在缓冲区溢出漏洞
- 并非所有缓冲区溢出漏洞都可被利用。
 - **各项要素必须对齐：** 内存布局、变量位置和跳转地址等必须刚好满足攻击条件，攻击才能成功。
- 如果漏洞可利用，攻击者可以注入代码
- 可能需要反复试验（试错）
 - 网上有大量帮助资源
- 栈溢出是“十年一度的顶级攻击”
 - 此外还有堆溢出、整数溢出、格式化字符串漏洞等

https://inst.eecs.berkeley.edu/~cs161/archive/fa08/papers/stack_smashing.pdf

缓冲溢出例子

- 假设程序要求输入一个 Trudy 不知道的序列号。
- 此外, Trudy **没有**源代码。
- Trudy 只有可执行文件 (exe) 。

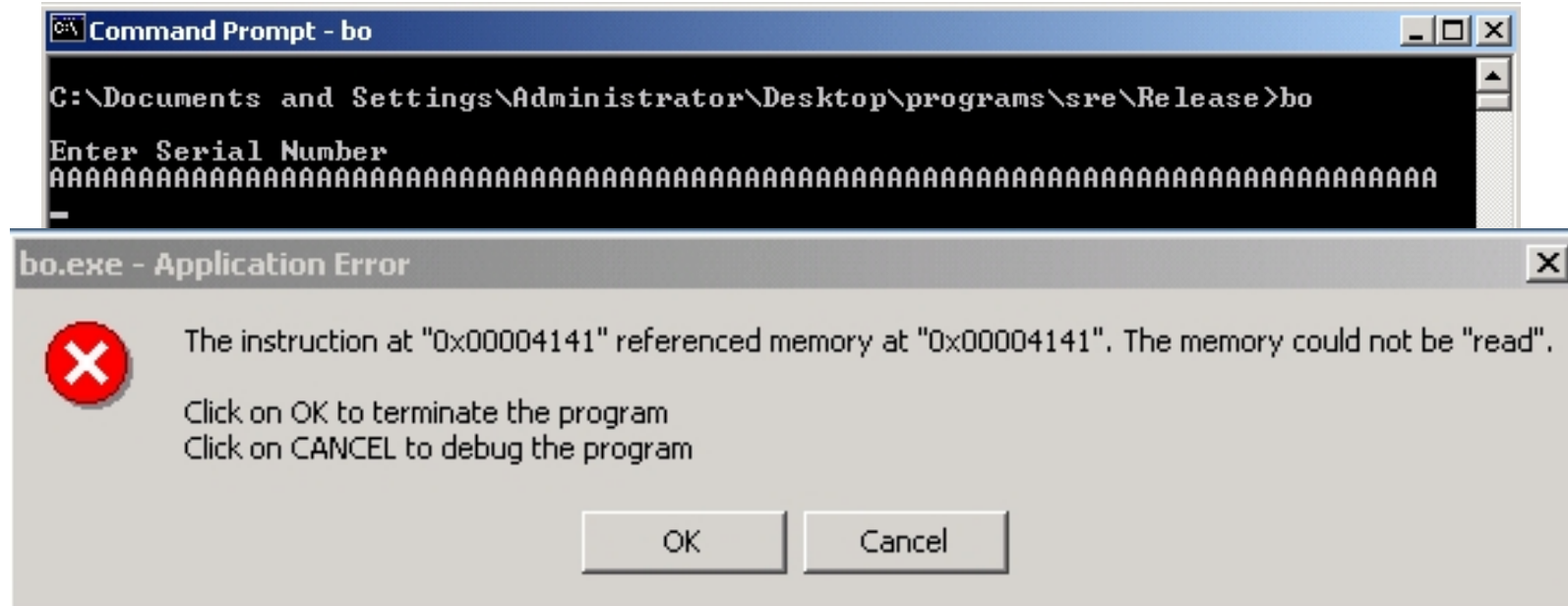


```
Command Prompt
C:\Documents and Settings\Administrator\Desktop\programs\sre\Release>bo
Enter Serial Number
woeiweiw
C:\Documents and Settings\Administrator\Desktop\programs\sre\Release>_
```

❑ 程序因序列号错误而退出

是否存在缓冲区溢出

- 通过反复试验，特鲁迪发现了明显的缓冲区溢出问题。



程序试图执行
位于内存地址
0x4141 处的指
令，但失败了。

因为 0x4141 通
常是一个无效的
或者受保护的内存区域，系统发现程序在“乱跑”，于是强制弹窗报错并关闭程序。

- ❑ 请注意， 0x41是“A”的 ASCII 码。
- ❑ **ret**似乎被覆盖了 2 个字节！

反汇编代码

- 接下来，反汇编bo.exe以查找

```
.text:00401000
.text:00401000
.text:00401003
.text:00401008
.text:0040100D
.text:00401011
.text:00401012
.text:00401017
.text:0040101C
.text:0040101E
.text:00401022
.text:00401027
.text:00401028
.text:0040102D
.text:00401030
.text:00401032
.text:00401034
.text:00401039
.text:0040103E

sub     esp, 1Ch
push    offset aEnterSerialNum ; "\nEnter Serial Number\n"
call    sub_40109F
lea     eax, [esp+20h+var_1C]
push    eax
push    offset aS              ; "%5"
call    sub_401088
push    8
lea     ecx, [esp+2Ch+var_1C]
push    offset aS123n456 ; "S123N456"
push    ecx
call    sub_401050
add     esp, 18h
test    eax, eax
jnz     short loc_401041
push    offset aSerialNumberIs ; "Serial number is correct.\n"
call    sub_40109F
add     esp, 4
```

- 目标是利用缓冲区溢出跳转到地址0x401034。

不想费劲去破解序列号算法，我
只想让程序**直接跳过**验证环节



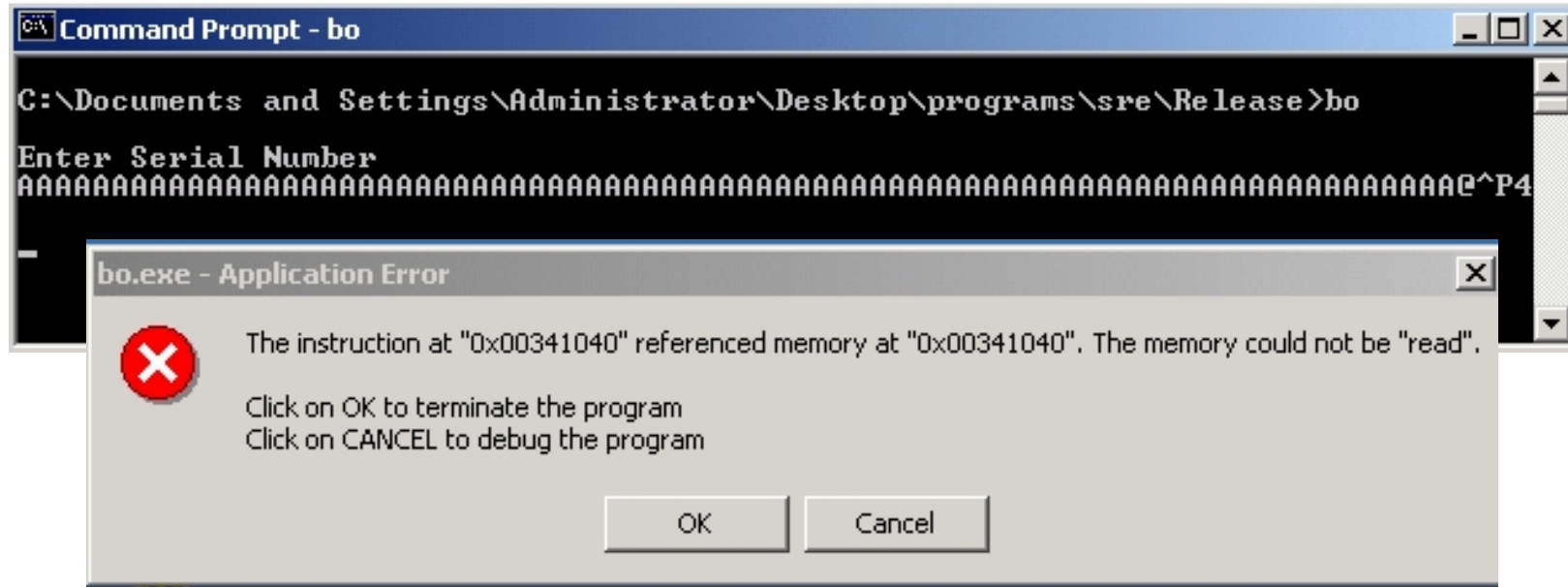
我们用一串 A 把程序搞崩溃
了，让它跳到了 0x4141。



我们要通过精确计算，把那串
A 替换成 **0x34 0x10 0x40**
(内存中的十六进制排列)。

缓冲区溢出攻击

- 发现0x401034在 ASCII 码中是“ @^P4 ”。



«» C

```
int x = 0x12345678;
```

地址从低到高:

地址	内容
0x1000	→ 78
0x1001	→ 56
0x1002	→ 34
0x1003	→ 12

- ❑ 字节顺序颠倒了？搞什么鬼.....？
- ❑ x86处理器是“小端存储”的。

0x401034

0x341040

缓冲区溢出攻击2

- 将字节顺序反转为“4^P@”，然后.....

缓冲区溢出

- Trudy **不需要** 访问源代码
- 唯一使用的工具是反汇编器，用于确定跳转到的地址。
- 通过反复尝试找到所需地址？
 - 如果攻击者没有可执行文件，则此步骤是必要的。
 - 例如，远程攻击

源代码

- 缓冲区溢出示例的源代码

- 攻击者很容易利用此漏洞.....

-却无法获取源代码!

```
#include <stdio.h>
#include <string.h>

main()
{
    char in[75];
    printf("\nEnter Serial Number\n");
    scanf("%s", in);

    if(!strcmp(in, "S123N456", 8))
    {
        printf("Serial number is correct.\n");
    }
}
```

堆栈溢出防范

- 采用**不可执行堆栈**
 - “禁止执行” **NX 位** (如果可用)
 - 这看起来合乎逻辑, 但有些实际代码是在堆栈上执行的 (例如 Java) 。
- 使用**金丝雀**
- 地址空间布局随机化 (**ASLR**)
- 使用**安全的编程语言** (Java、C#)
- 使用**更安全的 C 函数**
 - 对于不安全的函数, 存在更安全的版本。
 - 例如, 使用 `strncpy` 而不是 `strcpy`

堆栈溢出防范

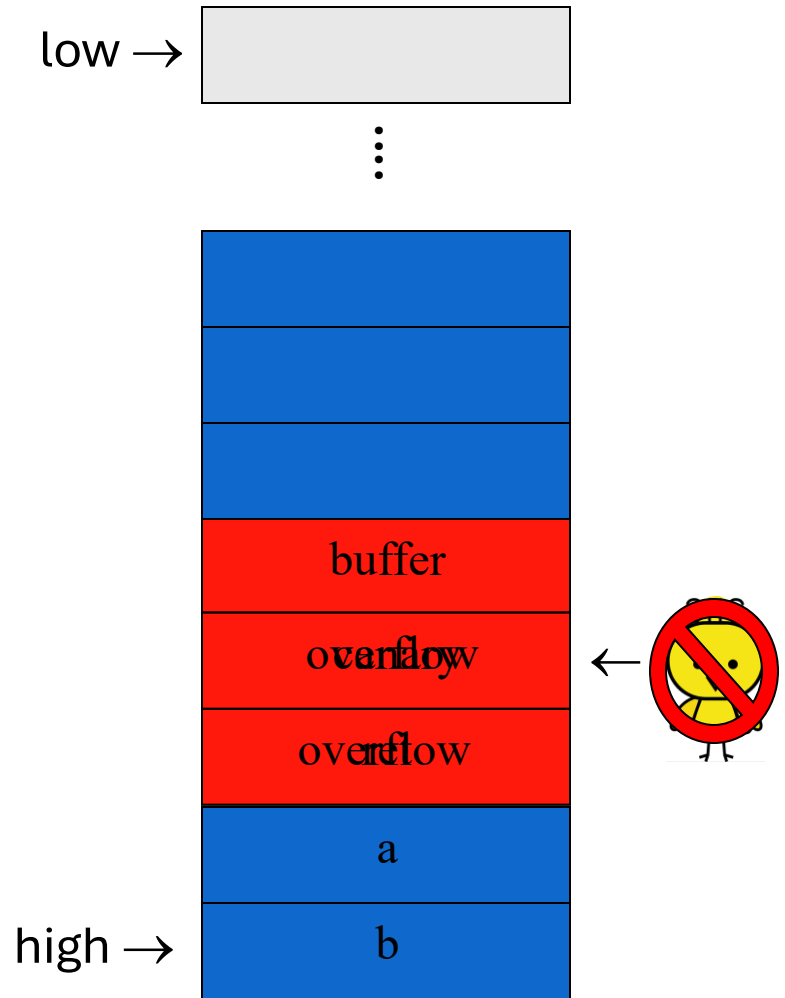
• 金丝雀

- 运行时堆栈检查
- 将金丝雀推到堆栈上
- 金丝雀值：
 - 常量0x000aff0d
 - 或者，可能取决于ret

在 C 语言等编程语言中，0x00（也叫 Null Byte）代表字符串的终点。

像 `strcpy` 这样的函数在拷贝字符串时，一旦遇到 0x00 就会停止。

攻击者为了不触发警报，必须把 0x000aff0d 原样写回内存。
攻击者无法通过输入字符串把 0x00 发送给程序。一旦他的输入里包含 0x00，传输就提前中断了，他根本没法完成后续地址的改写。



微软的 Canary 测试

- 微软通过/GS编译器标志为C++添加了**缓冲区安全检查功能**。
 - 基于金丝雀（或“安全 cookie”）

问： 金丝雀死后该怎么办？

A： 调用用户提供的“异常处理程序”来接管（报错，安全关闭）

- 弱点： 处理程序本身容易受到攻击
 - 攻击者可以指定处理程序代码
 - 使用/GS时， 以前“安全”的缓冲区溢出就会变得可被利用！

- 结果：
1. 金丝雀死了， 系统惊慌失措准备报警。
 2. 系统去寻找“报警电话”（处理程序指针）。
 3. 结果这个电话已经被攻击者换成了**恶意代码的地址**。
 4. 系统拨通了电话——**Boom!** 恶意代码执行了。

地址空间布局随机化 ASLR

Address Space Layout Randomization
(地址空间布局随机化)。

- 地址空间布局随机化
 - 随机化代码在内存中的加载位置
- 使得大多数缓冲区溢出攻击具有概率性。
- Windows Vista 使用了 256 种随机布局
 - 所以缓冲区溢出的概率大约是 $1/256$ 。
- Mac OS X 和其他操作系统中也有类似的情况。
- 针对微软ASLR的攻击确实存在。
 - 可以“去随机化”

内存泄漏 (Information Leak)

暴力破解

ROP (返回导向编程)

不完整验证

- 请考虑: `strcpy(buffer , argv[1])`
- 如果发生参数长度大于缓冲区长度, 则会发生缓冲区溢出。
`len(buffer ) < len(argv[1])`
- 软件必须通过检查`argv[1]`的长度
- 未能做到这一点, 体现了一个更普遍的问题: **验证不完整。**

输入验证

- 考虑网页表单数据
- 假设输入在客户端进行验证。
- 例如，以下说法是有效的。

`http://www.things.com/orders/final&custID=112&num=55A&qty=20&price=10&shipping=5&total=205`

- 假设服务器未对输入进行检查。
 - 既然客户端已经检查过输入，为什么还要费这个劲呢？
 - 然后攻击者就可以发送HTTP消息了。

`http://www.things.com/orders/final&custID=112&num=55A&qty=20&price=10&shipping=5&total=25`

不完整验证

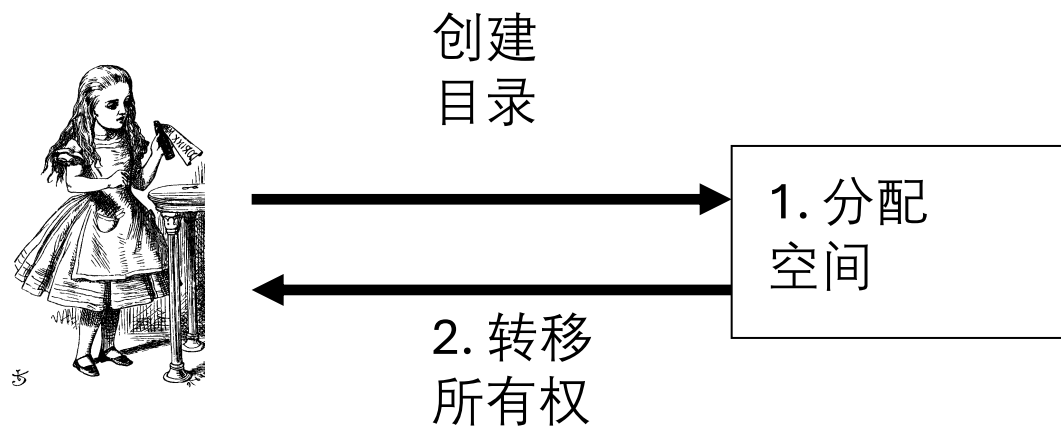
- Linux 内核
 - 研究发现了很多缓冲区溢出问题。
 - 其中很多都是由于验证不完整造成的。
- Linux 内核是“好的”软件， 因为
 - 开源
 - 内核——由编码大师编写
- 现有工具可以帮助我们发现这类问题。
 - 但验证不完整造成的错误可能很隐蔽。
 - 以及对攻击者也有用的工具！

竞态条件

- 竞态条件：攻击者与进程的下一个阶段操作之间的竞争。
- 安全流程应具有**原子性**。
 - “一次同时发生”
- 当安全关键流程分阶段进行时，可能会出现竞态条件。
- 攻击者在不同阶段之间进行更改
 - 通常，在授予授权的阶段之间，但在转移所有权的阶段之前。
- 例如： Unix mkdir

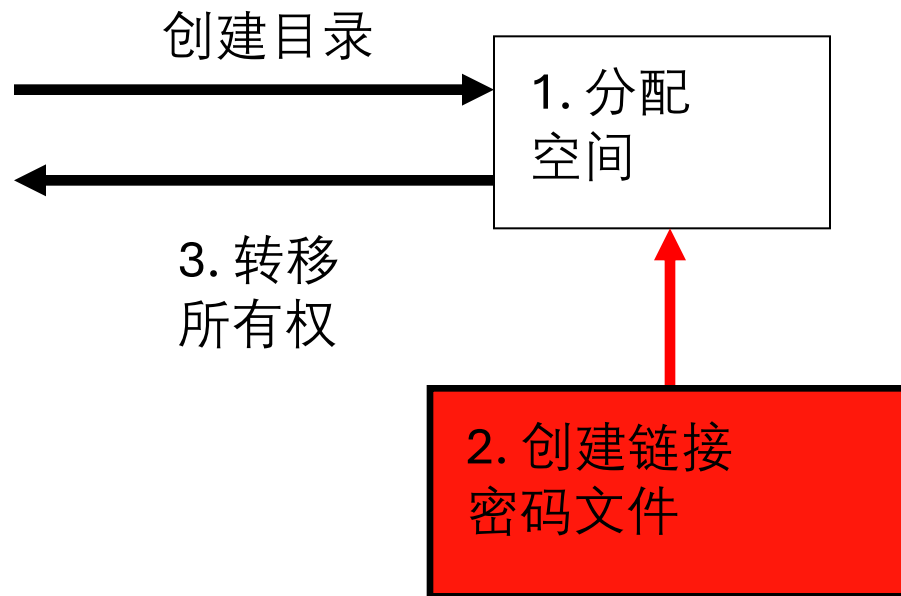
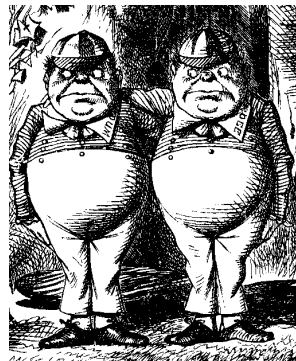
mkdir竞态条件

- ❑ mkdir创建新目录
- ❑ mkdir 命令应该如何工作



mkdir攻击

❑ mkdir 竞态条件



- ❑ 并非真正的“竞争”
 - 但攻击者选择的时机至关重要。

竞态条件

- 竞态条件很常见
- 竞态条件可能比缓冲区溢出更常见。
- 但竞态条件攻击需要严格的时间把握。
 - 如今缓冲区溢出漏洞很容易被利用。
- 为防止出现竞态条件，应使安全临界处理进程原子化。
 - 一次性发生，而不是分阶段发生
 - 在实践中并非总是容易实现的。

恶意软件

- 恶意软件并非新鲜事物.....
 - 弗雷德·科恩在 20 世纪 80 年代的开创性病毒研究
- 恶意软件类型（无标准定义）
 - **传播类 (Spreading):** 病毒、蠕虫（强调它们的扩散性）。
 - **控制类 (Stealth & Access):** 木马、后门、僵尸网络（强调隐蔽控制）。
 - **变现类 (Monetization):** 间谍软件、勒索软件（强调黑产盈利）。

病毒: 依赖宿主。

蠕虫: 自主传播。

木马: 欺骗用户。

勒索: 加密文件。

恶意软件

- 现在的恶意软件已经不是简单的破坏程序，而是变成了高度工程化，智能化的“攻击套件”。

传统恶意软件 (你的 PPT)	现代恶意软件 (最新趋势)
病毒/蠕虫 ：为了传播而传播	信息窃取 ：为了偷 Cookie/凭证
木马 ：简单的伪装功能	AI 深度伪造 ：社交工程自动化
逻辑炸弹 ：定时破坏	勒索软件 3.0 ：数据绑架与商业勒索
需要落地文件	无文件攻击 (Fileless) ： 只存在于内存

病毒在哪里藏身？

- 它们几乎生活在任何地方，例如.....
- **引导扇区**
 - 掌控开机权
- **内存居民**—幽灵般的存在
 - （无文件攻击）不保存在硬盘上，只在内存里。运行时杀毒软件找不到它
- **应用程序、宏、数据等**---最常见的伪装
- **库例程**（DLL/Shared Libraries）
- **编译器、调试器、病毒检查器等**—工具本身有毒
 - 所有编译出来的程序都自带后门。杀毒软件被感染了，整个防御体系就崩塌了。

SolarWinds 供应链攻击

恶意软件时间线

- 启蒙时代 (1980s) 概念验证
 - 脑病毒 (Brain, 1986): 世界上第一个PC病毒
 - 莫里斯蠕虫 (Morris Worm, 1988) : 互联网第一个蠕虫
- 混乱时代 (2000s) 效率与规模
 - 红色警报 (Code Red) & SQL Slammer: 极速传播
 - 僵尸网络 (Botnets) : 黑客追求控制权
- 武器化时代 (2010-) 精准打击与变现
 - 震网病毒 (Stuxnet, 2010) : 让伊朗核设施离心机停转
 - 勒索软件 (Ransomware) : 2013 年 **CryptoLocker** 出现, 结合了比特币支付和高强度加密, 开启了现代勒索产业。

80年代: “看看我多有才” (恶作剧/实验)



2000年代: “看看我传多快” (纯粹的破坏/带宽占用)



2010年以后: “要么给我钱, 要么我要命”

脑病毒

- 脑病毒 (Brain Virus) —— 世界上第一个 PC 病毒
- 时间：1986年
- 开发者：巴基斯坦，阿尔维兄弟
- 攻击目标：IBM PC（运行**MS-DOS** 系统）
- 工作模式：
 - 引导扇区病毒
 - 伪造内存数据
 - 自我复制
- 为了防止他们的软件盗版



病毒代码中竟公然写着他们的名字、电话和详细住址：
"Welcome to the Dungeon... Beware of this virus... Contact us for vaccination!"

“友好的”警告：它不删除数据，只是占用内存并修改卷标为 "**©Brain**"。

莫里斯蠕虫

初衷：只是为了通过
测量互联网的大小来
“满足好奇心”



- 莫里斯蠕虫：互联网安全时代的“成人礼”
- 时间：1988年
- 开发者：康奈尔研究生-罗伯特.泰潘.莫里斯
- 传播：互联网历史上第一个通过网络大规模自我传播的蠕虫。
- 波及范围：几个小时感染了互联网约10%的服务器。
- 工作模式：
 - 利用Unix系统的漏洞
 - 利用邮件程序调试模式获取远程控制权
 - 通过早期缓冲区溢出攻击技术，想查询指纹服务的端口注入恶意代码。
 - 字典攻击用户账户密码

PC Cyborg 木马

披着羊皮的狼

勒索软件的鼻祖

- PC Cyborg木马：最早的符合木马定义的案例
- 时间：1989年
- 伪装手段：伪装成一个关于艾滋病信息的调查问卷。
- 真实行为：一旦用户运行，它会加密用户文件并要求向巴拿马的一个邮政邮箱寄送赎金。
- 后期发展与演变：僵尸网络（Botnets）和勒索软件（Ransomware）



使用的对称加密，
密码就写在代码里

红色警报 (Code Red)

溢出漏洞可以演变为政治攻击和 **DDoS** 武器

- 红色警报：第一个真正意义上的全球性威胁，名字来源**樱桃汽水**
- **时间：2001年**
- 攻击对象：运行微软IIS（互联网信息服务）的服务器
- 攻击方式：利用缓冲区溢出漏洞。黑客发送一串长得离谱的字符直接淹没服务器的内存，改写执行指令。自动传播，不需要载体
- 阶段性行为：
 - 疯狂传播：寻找并感染其他服务器
 - 视觉破坏：把网页改成：“Welcome to <http://www.worm.com>! Hacked by Chinese!”
 - 终级大招：它设定了特定的日期，发动成千上万台服务器同时向**美国白宫官网**发起 DDoS 攻击，导致白宫不得不临时更换 IP 地址。

SQL Slammer

只有 **376 字节**

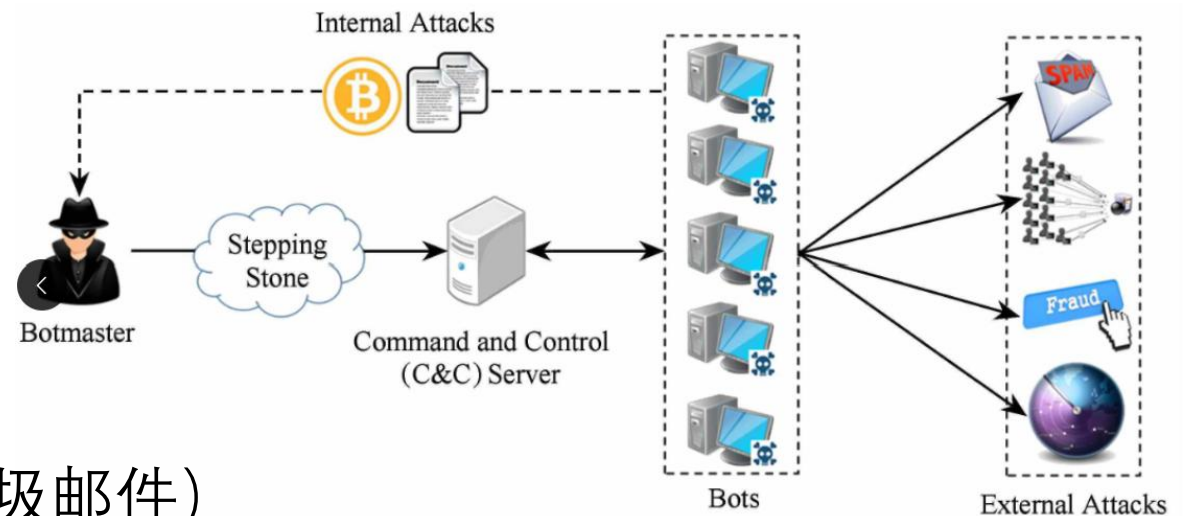
极简代码 + 高速网络可以
瞬间拖垮全球基础设施

- SQL Slammer: “瞬移”，历史上传播速度最快的蠕虫
- 时间: 2003年
- 攻击对象: 微软的SQL Server 2000 数据库
- 恐怖的速度: 10分钟内感染了全球绝大多数受害主机。每8.5秒，受感染的主机数量就翻一倍。
- 无感传播: 不需要文件，直接驻留在内存中。不需要确认接收，只管疯狂地往外发射带毒数据包
- 后果: 韩国互联网权限崩溃，美国ATM机取不出钱，航空订票瘫痪

僵尸网络

- 僵尸网络是指一类攻击者通过木马或者蠕虫控制成千上万台联网设备执行恶意活动。

- 主要活动：
 - DDoS攻击（让网站宕机）
 - 垃圾邮件分发（每天发送数十封垃圾邮件）
 - 算力窃取**（挖矿、暴力破解他人密码）



典型案例：Mirai 僵尸网络 (2016)。

震网病毒 (Stuxnet)

- **震网病毒**: 第一个国家级网络武器
- **时间**: 2010年
- **目标**: 伊朗纳坦兹 (Natanz) 核设施的铀浓缩离心机。
- **物理破坏**: 它的目的不是偷资料, 也不是瘫痪电脑, 而是让离心机**物理炸裂**。
- **手段**: 它潜入控制系统 (SCADA), 悄悄改变离心机的转速 (忽快忽慢), 导致设备由于产生共振而金属疲劳并报废。

恶意软件检测

- 恶意软件检测技术
 - 特征检测
 - 变化检测
 - 异常检测
 - 机器学习（深度学习、人工智能等）
- 我们将简要讨论这些内容。

特征值检测

- 签名可能是 exe 文件中的一串比特。
 - 也可以使用通配符、哈希值等。

- 例如, W32/Beast病毒具有特征码。

83EB 0274 EB0E 740A 81EB 0301 0000

- 也就是说, 这串比特出现在病毒中。
- 我们可以在所有文件中搜索此签名
- 如果找到了字符串, 我们是否找到了 W32/Beast?
 - 不一定——字符串可能在普通代码中。
 - 随机情况下, 概率只有 $1/2^{112}$
 - 但软件并非随机生成.....

特征值检测

- 优势
 - 对“普通”或“经典”恶意软件有效
 - 用户/管理员负担最小
- 缺点
 - 签名文件可能很大（数万个字符）导致扫描速度变慢
 - 签名文件必须保持最新状态。
 - **无法检测到未知病毒**
 - 无法检测到某些高级类型的恶意软件。
- 最常用的检测方法

变化检测

- 病毒必定存在于某个地方。
- 如果您发现某个文件发生了意外更改，则该文件可能已被感染。
- 如何检测变化？
 - 对文件进行哈希处理并（安全地）存储哈希值
 - 定期重新计算哈希值并进行比较
 - 如果哈希值发生变化，文件**可能**已被感染。

变化检测

- 优势
 - 几乎没有（或根本没有）假阴性
 - 能够检测到以前未知的恶意软件
- 缺点
 - 许多文件会发生变化——而且经常如此。
 - 可能出现很多误报（假阳性）
 - 用户/管理员负担沉重
 - 如果检测到可疑变化，该怎么办？ **可能不得不依赖特征码检测。**

异常检测

- 监控系统，查找任何“异常”、“类似病毒”或“潜在恶意”或其他可疑情况.....
- 异常事物的例子
 - 文件以某种意想不到的方式发生了变化。
 - 系统出现某种异常行为
 - 意外的网络活动
 - 意外的文件访问等等。
- 但是，我们首先必须定义“正常”。
 - 而正常状态会（而且必须）随着时间而改变

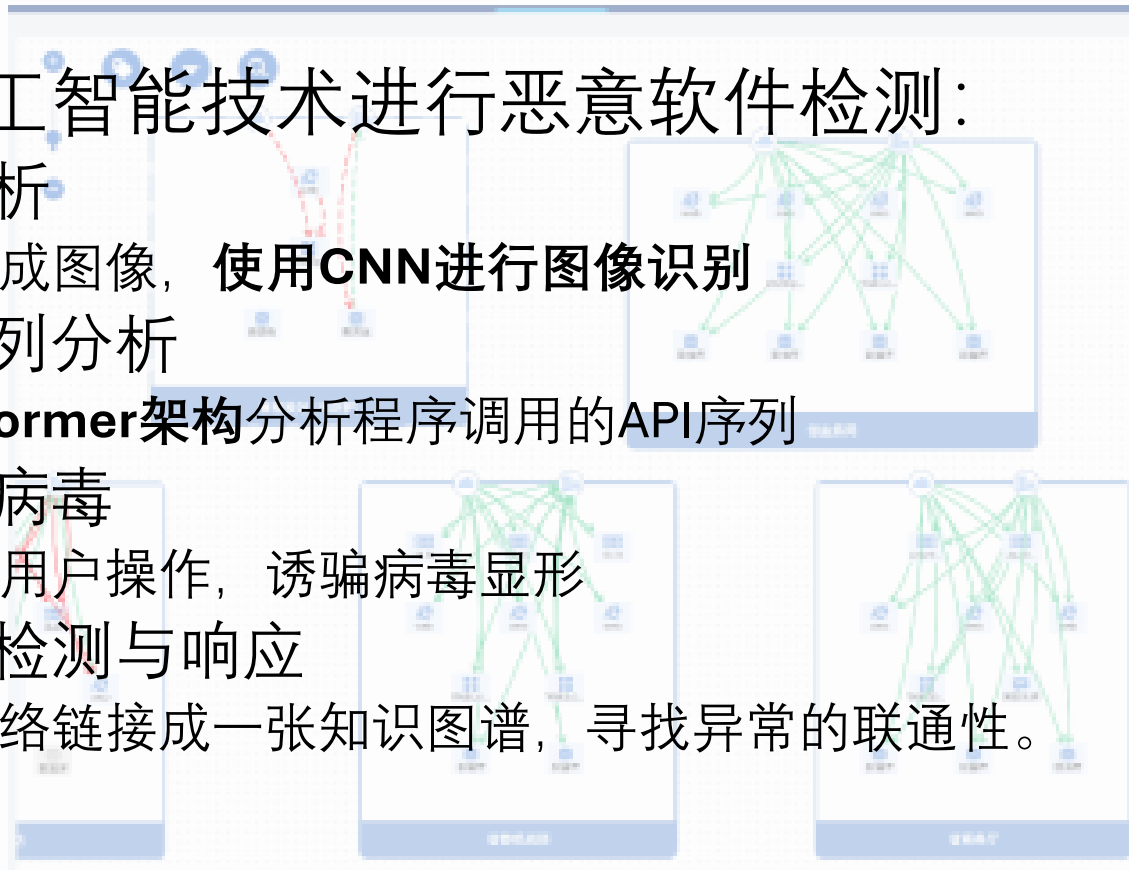
异常检测

- 优势
 - 检测到未知恶意软件的几率
- 缺点
 - 过往记录令人质疑
 - 特鲁迪可能会把不正常的事情伪装成正常的事情（慢慢来）
 - 通常，异常检测会与其他方法（例如，特征检测）结合使用。
- 在入侵检测（IDS）中也很受欢迎
- 棘手的未解决（或无法解决？）问题
 - 这听起来像是人工智能（或者说机器学习、深度学习、大数据）的工作。

基于AI的恶意软件检测

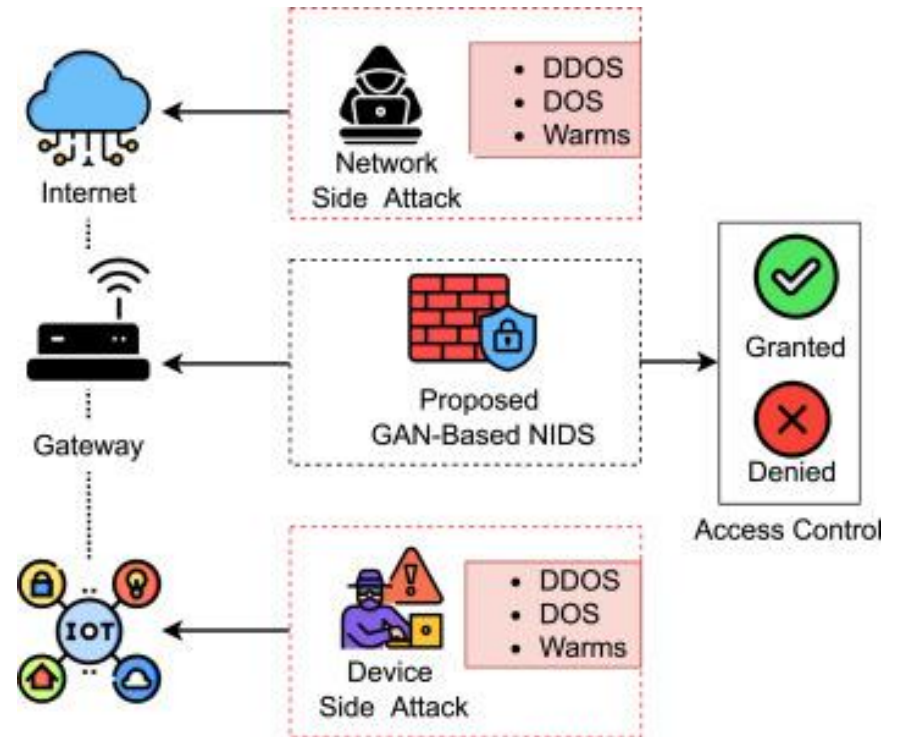
流量可视化

- 基于机器学习和人工智能技术进行恶意软件检测：
 - 基于特征的静态分析
 - 将文件二进制转化成图像，使用**CNN**进行图像识别
 - 基于行为的动态序列分析
 - 使用**RNN**或**Transformer**架构分析程序调用的API序列
 - 基于强化学习检测病毒
 - 利用**强化学习**模拟用户操作，诱骗病毒显形
 - 基于**图分析**的终端检测与响应
 - 将内网，文件和网络链接成一张知识图谱，寻找异常的联通性。



软件安全

- 我们不断地进行升级检测恶意软件地方法。
- 攻击者也在不断地去寻找方法躲过我们地防御系统。
- 我们发明了**特征码**，黑客就发明了**多态变形**；
- 我们发明了**变化检测**，黑客就转入**内存潜伏**；
- 现在我们引入了**机器学习**，黑客也开始用 **生成式 AI** 来对抗我们的模型。



<https://www.sciencedirect.com/science/article/pii/S2542660524001537>

Thank you.